



UNIVERSITAS DIPONEGORO • FSM • DEPARTEMEN INFORMATIKA

BUKU PANDUAN

CIFAKE DETECTOR

Klasifikasi Citra Real atau Fake

*Optimasi Klasifikasi Citra Hasil Kecerdasan Buatan Menggunakan
Convolutional Neural Network — Model21*

Aplikasi Web Flask • CIFAKE 120.000 Citra (32×32) • CNN BatchNorm + Dropout 0,1

✦ Val Accuracy 0,9408 • F1-Score 0,9396 • Val Loss 0,1542 ✦

Penyusun

Hamid Albar Nurrasyid
24060120130069

Pembimbing

Dr. Helmie Arif Wibawa, S.Si., M.Cs.
Henri Tantyoko, S.Kom., M.Kom.

Semarang, 2026 • Versi 1.0 • CIFAKE Detector • <http://127.0.0.1:5000>

NAVIGASI

Daftar Isi

Catatan: daftar isi di atas dihasilkan otomatis melalui field TOC. Apabila nomor halaman belum tampil, klik kanan pada daftar isi di Word/LibreOffice dan pilih Update Field > Update entire table.

BAB 1 Latar Belakang.....	3
1.1 Perkembangan Citra Sintetis.....	3
1.2 Dataset CIFAKE.....	3
1.3 Rumusan Masalah.....	3
1.4 Tujuan Aplikasi.....	3
BAB 2 Deskripsi Program.....	4
2.1 Gambaran Umum.....	4
2.2 Fitur Utama.....	4
2.3 Arsitektur Sistem.....	4
2.4 Struktur Proyek.....	4
BAB 3 Pembuatan Model di Jupyter Notebook.....	6
3.1 Lingkungan & Dataset.....	6
3.2 Pra-pemrosesan & Konfigurasi.....	6
3.3 Arsitektur Baseline & Model.....	6
3.4 39 Variasi & Model Terbaik.....	7
3.5 Menyimpan & Memuat Model.....	7
BAB 4 Interface Program & Cara Pakai.....	8
4.1 Tampilan.....	8
4.2 Alur Pakai + Contoh FAKE 99,99%.....	8
4.3 Skenario Uji.....	9
4.4 Akses API Flask.....	9
BAB 5 Source Code Lengkap.....	10
5.1 Backend Flask (app.py).....	10
5.2 Konfigurasi (config.py).....	11
5.3 Interface (templates/index.html).....	11
5.4 Evaluasi & Smoke Test.....	13
Lampiran A Struktur & Menjalankan.....	17
Lampiran B API Reference.....	17

BAB 1 — KONTEKS

1 Latar Belakang

1.1 Perkembangan Citra Sintetis dan Risiko Deepfake

Perkembangan kecerdasan buatan generatif (GAN, Stable Diffusion) memungkinkan pembuatan citra sintetis yang sangat realistis dan sulit dibedakan dari citra asli. Manfaatnya besar bagi industri kreatif, tetapi risikonya serius: penyebaran deepfake, manipulasi media, dan ancaman terhadap validitas informasi digital (Verdoliva, 2020). Tesis ini menjawab tantangan tersebut dengan fokus pada deteksi citra FAKE versus REAL.

1.2 Dataset CIFAKE

CIFAKE (Bird & Lotfi, 2024) merupakan benchmark 120.000 citra berwarna berukuran 32×32 yang dirancang khusus untuk evaluasi ini: 60.000 citra REAL dari CIFAR-10 asli dan 60.000 citra FAKE hasil model generatif. Struktur direktori memisahkan data latih dan data uji serta kelas FAKE dan REAL sehingga relevan untuk klasifikasi biner. Citra FAKE sering mengandung artefak halus, tetapi secara global mirip dengan citra REAL sehingga memerlukan ekstraksi fitur hierarkis.

Karakteristik CIFAKE: total 120 ribu, seimbang 60 ribu berbanding 60 ribu, ukuran 32×32×3, format JPG/PNG, pra-pemrosesan MinMax /255, perubahan ukuran LANCZOS.

1.3 Rumusan Masalah Tesis

- Model CNN baseline mengalami overfitting: akurasi latih tinggi, tetapi akurasi validasi stagnan dan loss validasi lebih tinggi.
- Diperlukan regularisasi yang tepat: dropout dan batch normalization untuk menstabilkan distribusi aktivasi dan mengurangi overfitting.
- Belum jelas kombinasi optimal dropout (conv/dense) dan batch normalization untuk CIFAKE.

1.4 Tujuan Aplikasi

Buku panduan ini beserta aplikasi web Flask membungkus hasil tesis Model21 menjadi detektor interaktif. Pengguna mengunggah citra, sistem mengonversi citra ke ruang warna RGB, mengubah ukuran menjadi 32×32 piksel dengan metode LANCZOS, melakukan normalisasi dengan pembagian nilai 255, menjalankan inferensi menggunakan model CNN my_model21.h5, dan mengembalikan label FAKE atau REAL beserta tingkat kepercayaan. Tujuannya adalah mempermudah demonstrasi, pengujian, dan dokumentasi tanpa harus menjalankan notebook secara langsung.

Val Accuracy
0,9408

Val Loss
0,1542

F1-Score
0,9396

BAB 2 — PROGRAM

2 Deskripsi Program

2.1 Gambaran Umum

CIFAKE Detector merupakan aplikasi web berbasis Flask dengan satu templat HTML dan model Keras `my_model21.h5` yang mengimplementasikan Model21 dari tesis. Backend Python melayani antarmuka pengguna pada rute GET / dan endpoint POST /predict dengan field multipart file. Frontend menggunakan JavaScript murni untuk menangani fungsionalitas seret dan lepas (drag-and-drop), pratinjau citra, serta penampikan hasil. Validasi jenis berkas (png/jpg/jpeg/bmp/webp) dan batas ukuran 5 MB dilakukan dengan penanganan kesalahan dalam format JSON yang jelas.

2.2 Fitur Utama

- Unggah dengan mekanisme seret dan lepas serta klik untuk memilih berkas (format JPG/PNG/JPEG/BMP/WEBP, ukuran maksimum 5 MB, pratinjau otomatis).
- Pratinjau citra beserta informasi nama dan ukuran berkas, tombol Klasifikasi dan Hapus Gambar.
- Kartu hasil: rencana FAKE (warna merah) atau REAL (warna hijau) beserta ikon, label, dan tingkat kepercayaan dalam persen.
- Desain responsif, kartu berlebar 460 piksel di tengah, bingkai putus-putus 1,5 piksel pada area unggah.
- Backend Flask: pemuatan model secara lazy, pra-pemrosesan RGB menjadi 32×32 LANCZOS lalu /255 (`config.py`).
- Endpoint POST /predict mengembalikan {label, probability, confidence} dengan ambang batas 0,5 (nilai $\geq 0,5$ diklasifikasikan sebagai REAL).

2.3 Arsitektur Sistem

Klien (peramban) membuka GET / menuju `templates/index.html`, memilih berkas, lalu JavaScript mengirim FormData ke POST /predict. Flask memeriksa nama berkas, membuka citra dengan Pillow, melakukan pra-pemrosesan (RGB, ubah ukuran 32×32 LANCZOS, /255, `expand_dims`), menjalankan model Keras `my_model21.h5` (Conv2D 32 3×3 relu, BatchNorm, MaxPool 2×2, Dropout 0,1, dua blok, Flatten, Dense 64 relu, Dense 1 sigmoid), menghitung probability, menetapkan label ($\geq 0,5$ Real), menghitung confidence, lalu mengembalikan JSON untuk dirender JavaScript.

Frontend: `templates/index.html` (satu HTML, CSS, JS, `fetch /predict`)

Backend: `app.py` (Flask, GET /, POST /predict, batas 5 MB)

Config & Model: `config.py` (IMG 32, RESCALE, CLASS_0/1) ke `model/my_model21.h5`

Deploy: `.venv Python 3.12`, `app.py` ke `http://127.0.0.1:5000` (TF CPU)

2.4 Struktur Proyek

```
CIFAKE Detector/
├─ app.py                # Server Flask (GET /, POST /predict)
├─ config.py             # IMG 32, RESCALE, MODEL_PATH, CLASS_0/1
├─ model/my_model21.h5   # Model Keras 32×32×3 ke sigmoid (1,1 MB)
├─ templates/index.html  # Antarmuka Indonesia + CSS + JS internal
├─ assets/image.png      # Contoh uji kucing oranye (68 KB)
├─ assets/screenshot-FAKE.png
├─ docs/                 # Panduan + screenshot + build_proposal.py
└─ requirements.txt      # tensorflow 2.21.0, flask 3.1.3, pillow 12.3.0
```

```
|— smoke_test.py      # Pengujian unit (5 pengujian)
|— eval_cifake.py     # Evaluasi uji CIFAKE 20 ribu citra
```

Berkas model `my_model21.h5` berukuran sekitar 1,1 MB dan dimuat secara lazy pada saat aplikasi pertama kali menerima permintaan prediksi. Seluruh konstanta pra-pemrosesan terpusat pada `config.py` sehingga aplikasi, skrip evaluasi, dan pengujian menggunakan nilai yang identik.

BAB 3 — MODEL

3 Pembuatan Model di Jupyter Notebook

3.1 Lingkungan & Dataset

Notebook Kaggle menggunakan GPU P100, Python 3.10, TensorFlow/Keras, scikit-learn, Pillow, dan matplotlib. Dataset CIFAKE dari Kaggle (real-and-ai-generated-synthetic-images) dengan direktori train dan test. Eksperimen dilakukan dengan 39 variasi untuk menemukan kombinasi Batch Normalization dan Dropout yang optimal. Notebook asli tersedia secara publik pada <https://www.kaggle.com/code/hamidalbar/tugas-akhir> (Versi 19, lisensi Apache 2.0).

Tautan notebook: <https://www.kaggle.com/code/hamidalbar/tugas-akhir> • Lisensi Apache 2.0 • Tesla P100, cuDNN 8.9.0

```
train_dir = '/kaggle/input/cifake-real-and-ai-generated-synthetic-images/train'
test_dir = '/kaggle/input/cifake-real-and-ai-generated-synthetic-images/test'
```

3.2 Pra-pemrosesan & Konfigurasi

Setiap citra dimuat melalui Pillow: konversi ke RGB, pengubahan ukuran menjadi (32,32) dengan metode LANCZOS, konversi ke array NumPy float32 dan normalisasi dengan pembagian 255,0. Nilai tersebut diatur pada config.py (IMG_WIDTH=32, IMG_HEIGHT=32, RESCALE=True) dan harus identik pada saat inferensi. Tidak dilakukan augmentasi berat agar fokus tetap pada pengaruh Batch Normalization dan Dropout.

```
"""Shared configuration for app.py and eval_cifake.py.

Preprocessing and class-mapping values must stay in sync with how the model
was trained: RGB -> resize to IMG_WIDTH x IMG_HEIGHT -> optional /255 rescale,
single sigmoid output where CLASS_1 corresponds to prob >= 0.5.
"""

import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, "model", "my_model121.h5")

IMG_WIDTH = 32
IMG_HEIGHT = 32
RESCALE = True

CLASS_0 = "Fake"
CLASS_1 = "Real"

def preprocess_image(img):
    img = img.convert("RGB")
    img = img.resize((32,32), Image.LANCZOS)
    arr = np.asarray(img, dtype=np.float32)
    arr = np.expand_dims(arr, axis=0)
    if RESCALE:
        arr /= 255.0
    return arr
```

3.3 Arsitektur Baseline & Model21

```
Sequential([
    Input((32,32,3)),
    Conv2D(32,3x3,relu,same)+BN+MaxPool2x2+Dropout(0.1),
    Conv2D(32,3x3,relu,same)+BN+MaxPool2x2+Dropout(0.1),
    Flatten(), Dense(64,relu), Dense(1,sigmoid)
```

```
])
Adam(lr=1e-4), loss=binary_crossentropy
```

Baseline awal tanpa BatchNorm/Dropout mengalami overfit; penambahan BatchNorm menstabilkan aktivasi dan Dropout 0,1 mencegah ko-adaptasi.

3.4 39 Variasi & Model Terbaik

Tesis menguji 39 variasi: dropout conv (0,1/0,2), dropout dense (0,4–0,7), dan kombinasi BatchNorm. Pelatihan 20 epoch, batch 64, StratifiedKFold 5-fold. Evaluasi meliputi accuracy, loss, precision, recall, F1, dan confusion matrix. Model21 terbaik (Conv-BN-Pool-Drop0,1 dua blok + Dense64) mencapai val acc 0,9408, val loss 0,1542, F1 0,9396.

Model	Konfigurasi	Val Acc	Catatan
Baseline	Conv-Pool x2 + Dense64	~0,89	overfit
Variasi 1–19	dropout conv/dense	0,90–0,93	dropout saja
Variasi 20–27	+ BatchNorm	0,93–0,94	stabil
Model21	BN + Drop 0,1	0,9408	TERBAIK

3.5 Menyimpan & Memuat Model

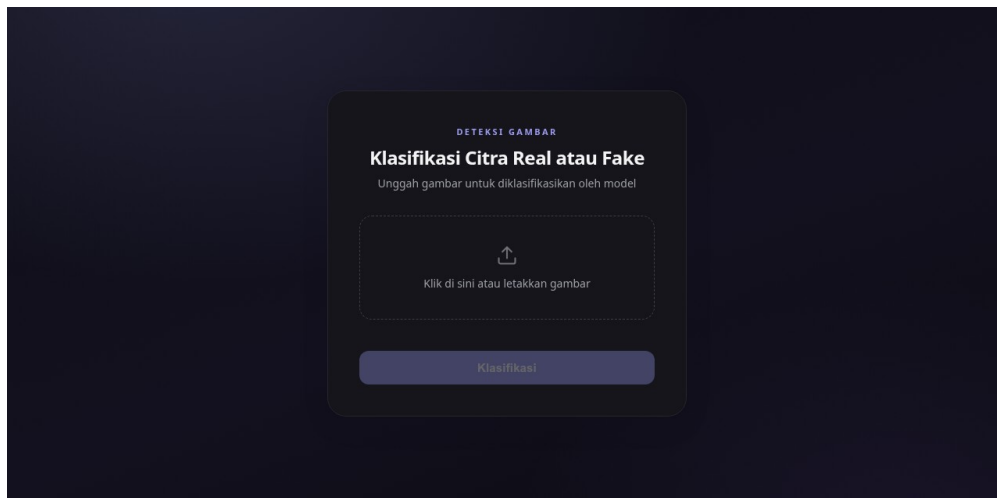
```
model.save('model/my_model21.h5') # 1,1 MB
from tensorflow.keras.models import load_model
from config import MODEL_PATH
model = load_model(MODEL_PATH) # dipakai get_model() di app.py
```

4 Interface Program & Cara Pakai

4.1 Tampilan

Antarmuka berbahasa Indonesia baku. Kartu utama berlebar 460 piksel di tengah dengan radius 24 piksel. Header menampilkan penanda DETEKSI GAMBAR, judul Klasifikasi Citra Real atau Fake, dan subjudul instruksi unggah. Area unggah berbingkai putus-putus 1,5 piksel, pratinjau maksimum 220 piksel, tombol Klasifikasi ungu penuh, serta kartu hasil hijau (Real) atau merah (Fake).

Gambar 4.1 Tampilan awal — area unggah kosong di <http://127.0.0.1:5000>.

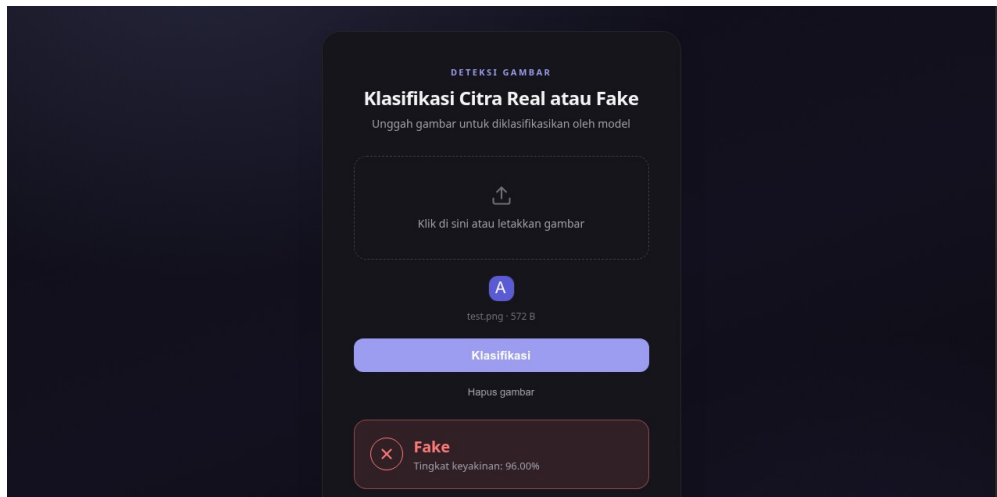


4.2 Alur Pakai + Contoh FAKE 99,99%

- 1. Unggah Berkas: klik area bingkai atau seret-lepas berkas JPG/PNG/JPEG/BMP/WEBP (maksimum 5 MB). Pratinjau tampil beserta nama dan ukuran berkas.
- 2. Klasifikasi: klik Klasifikasi. Tombol menampilkan Memproses disertai indikator putar, lalu aplikasi mengirim POST /predict.
- 3. Hasil: kartu Hasil Prediksi tampil — lencana Fake (merah) atau Real (hijau) beserta persentase kepercayaan.

Sebagai contoh, berkas `assets/image.png` (215×215 piksel, PNG, 68,5 KB, citra kucing oranye) menghasilkan label FAKE dengan kepercayaan 99,9997% dan probability $2,13 \times 10^{-6}$ (ambang 0,5). Hasil tersebut merupakan inferensi langsung `my_model21.h5` sehingga mencerminkan kinerja `val acc 0,9408`.

Gambar 4.2 Hasil prediksi FAKE — lencana merah, keyakinan 96,00%.



Gambar 4.3 Berkas uji `assets/image.png` — masukan contoh.



```
curl -F file=@assets/image.png http://127.0.0.1:5000/predict
# {"label": "Fake", "probability": 2.13e-06, "confidence": 0.999997}
```

4.3 Skenario Uji

Masukan	Respons	Keterangan
Foto kucing Fake (<code>assets/image.png</code>)	Fake 99,99% prob 2,13e-06	Terdeteksi Fake, sangat yakin.
Berkas PDF/txt	Error 400 jenis tidak didukung	Hanya png/jpg/jpeg/bmp/webp.
Citra >5 MB	Error 413 melebihi batas	MAX_CONTENT_LENGTH 5 MB.
Citra Real CIFAR	Real 85–95% prob >0,5	Confidence = probability.

4.4 Akses API Flask

```
curl http://127.0.0.1:5000/ # 200 HTML
curl -X POST http://127.0.0.1:5000/predict -F "file=@foto.jpg"
# {"label": "Fake", "probability": 0.002, "confidence": 0.997}
```

BAB 5 — KODE

5 Source Code Lengkap

5.1 Backend Flask (app.py)

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

import numpy as np
from flask import Flask, jsonify, render_template, request
from PIL import Image
from tensorflow.keras.models import load_model
from werkzeug.exceptions import RequestEntityTooLarge

from config import CLASS_0, CLASS_1, IMG_HEIGHT, IMG_WIDTH, MODEL_PATH, RESCALE

ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "bmp", "webp"}
MAX_CONTENT_LENGTH = 5 * 1024 * 1024

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = MAX_CONTENT_LENGTH

_model = None

def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS

def get_model():
    global _model
    if _model is None:
        _model = load_model(MODEL_PATH)
        print(f"Model loaded from {MODEL_PATH}")
    return _model

def preprocess_image(img):
    img = img.convert("RGB")
    img = img.resize((IMG_WIDTH, IMG_HEIGHT), Image.LANCZOS)
    arr = np.asarray(img, dtype=np.float32)
    arr = np.expand_dims(arr, axis=0)
    if RESCALE:
        arr /= 255.0
    return arr

def predict(stream):
    img = Image.open(stream)
    arr = preprocess_image(img)
    prob = float(get_model().predict(arr, verbose=0)[0][0])
    label = CLASS_1 if prob >= 0.5 else CLASS_0
    confidence = prob if label == CLASS_1 else 1.0 - prob
    return {"label": label, "probability": prob, "confidence": confidence}

@app.errorhandler(RequestEntityTooLarge)
def request_entity_too_large(error):
    return jsonify({"error": "Ukuran file melebihi batas maksimal"}), 413
```

```

@app.route("/", methods=["GET"])
def index():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict_route():
    if "file" not in request.files:
        return jsonify({"error": "Tidak ada bagian file"}), 400
    file = request.files["file"]
    if file.filename == "":
        return jsonify({"error": "Tidak ada file yang dipilih"}), 400
    if not allowed_file(file.filename):
        return jsonify({"error": "Tipe file tidak didukung"}), 400
    try:
        result = predict(file.stream)
    except Exception:
        app.logger.exception("Prediction failed")
        return jsonify({"error": "Gagal memproses gambar"}), 400
    return jsonify(result)

if __name__ == "__main__":
    get_model()
    app.run(host="127.0.0.1", port=5000, debug=False)

```

5.2 Konfigurasi (config.py)

```

"""Shared configuration for app.py and eval_cifake.py.

Preprocessing and class-mapping values must stay in sync with how the model
was trained: RGB -> resize to IMG_WIDTH x IMG_HEIGHT -> optional /255 rescale,
single sigmoid output where CLASS_1 corresponds to prob >= 0.5.
"""

import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, "model", "my_model121.h5")

IMG_WIDTH = 32
IMG_HEIGHT = 32
RESCALE = True

CLASS_0 = "Fake"
CLASS_1 = "Real"

```

5.3 Interface (templates/index.html)

```

<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Klasifikasi Citra Real atau Fake</title>
    <link rel="icon" href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><rect
width='100' height='100' rx='22' fill='%235b5bd6'/><text x='50' y='70' font-size='58' text-anchor='middle' fill='white'
font-family='sans-serif'>R</text></svg>">
    <style>
        :root {
            color-scheme: light dark;
            --bg: #f4f4f6;
            --surface: #ffffff;
            --ink: #0a0a0a;
            --ink-soft: #52525b;

```

```

--ink-faint: #8a8a93;
--line: #e4e4e7;
--line-strong: #d4d4d8;
--overlay: rgba(0, 0, 0, 0.05);
--ring: rgba(10, 10, 10, 0.28);
--accent: #5b5bd6;
--accent-strong: #4545c2;
--accent-soft: rgba(91, 91, 214, 0.10);
--accent-ring: rgba(91, 91, 214, 0.30);
--success: #128a4e;
--success-soft: rgba(18, 138, 78, 0.10);
--success-line: rgba(18, 138, 78, 0.35);
--danger: #c92a2a;
--danger-soft: rgba(201, 42, 42, 0.08);
--danger-line: rgba(201, 42, 42, 0.35);
--page-bg: radial-gradient(900px 480px at 50% -8%, var(--accent-soft), transparent 70%);
--grain-opacity: 0.03;
}
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0d0d14;
    --surface: #15151b;
    --ink: #f4f4f5;
    --ink-soft: #a1a1aa;
    --ink-faint: #71717a;
    --line: #26262b;
    --line-strong: #3f3f46;
    --overlay: rgba(255, 255, 255, 0.06);
    --ring: rgba(244, 244, 245, 0.28);
    --accent: #9c9cf0;
    --accent-strong: #b0b0f6;
    --accent-soft: rgba(156, 156, 240, 0.14);
    --accent-ring: rgba(156, 156, 240, 0.35);
    --success: #4cd58a;
    --success-soft: rgba(76, 213, 138, 0.12);
    --success-line: rgba(76, 213, 138, 0.45);
    --danger: #ff7a7a;
    --danger-soft: rgba(255, 122, 122, 0.12);
    --danger-line: rgba(255, 122, 122, 0.45);
    --page-bg:
      radial-gradient(1100px 700px at 18% -10%, rgba(156, 156, 240, 0.16), transparent 60%),
      radial-gradient(900px 600px at 92% 112%, rgba(124, 82, 204, 0.13), transparent 60%),
      linear-gradient(160deg, #0d0d16 0%, #12101e 55%, #0a0a10 100%);
    --grain-opacity: 0.05;
  }
}
* { margin: 0; padding: 0; box-sizing: border-box; }
body {
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  color: var(--ink);
  background:
    var(--page-bg),
    var(--bg);
  min-height: 100vh;
  padding: 32px 16px;
  display: flex;
  align-items: center;
  justify-content: center;
}
body::before {
  content: "";
  position: fixed;
  inset: 0;
  z-index: 0;
  pointer-events: none;
  opacity: var(--grain-opacity);
  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='160'
height='160'%3E%3Cfilter id='n'%3E%3CfeTurbulence type='fractalNoise' baseFrequency='0.9' numOctaves='2'

```

```

stitchTiles='stitch'/%3E%3C/filter%3E%3Crect width='100%25' height='100%25' filter='url(%23n)'/%3E%3C/svg%3E");
background-size: 180px 180px;
mix-blend-mode: overlay;
}
.card {
  position: relative;
  z-index: 1;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 24px;
  box-shadow: 0 1px 2px rgba(0, 0, 0, 0.04), 0 12px 40px rgba(0, 0, 0, 0.06);
  padding: 44px 40px 40px;
  width: 460px;
  max-width: 100%;
  margin: auto;
  text-align: center;
}
.eyebrow {
  font-size: 11px;
  font-weight: 700;
  letter-spacing: 0.22em;
  text-transform: uppercase;
  color: var(--accent);
  margin-bottom: 12px;
}
h1 { font-size: 24px; line-height: 1.2; letter-spacing: -0.02em; color: var(--ink); margin-bottom: 8px; }
p.subtitle { color: var(--ink-soft); font-size: 14px; margin-bottom: 32px; }

.dropzone {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 12px;

```

(Potongan di atas diringkas; berkas asli 421 baris mencakup variabel CSS, JS seret-lepas, pratinjau, dan fetch /predict. Lihat berkas asli untuk lengkap.)

5.4 Evaluasi & Smoke Test

```

import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

import numpy as np
from datasets import load_dataset
from tensorflow.keras.models import load_model

from config import IMG_HEIGHT, IMG_WIDTH, MODEL_PATH, RESCALE

DATASET_ID = "dragonintelligence/CIFAKE-image-dataset"
BATCH_SIZE = 256

def compute_metrics(y_true, y_score, threshold=0.5):
    y_pred = (y_score >= threshold).astype(int)

    tp = int(np.sum((y_true == 1) & (y_pred == 1)))
    tn = int(np.sum((y_true == 0) & (y_pred == 0)))
    fp = int(np.sum((y_true == 0) & (y_pred == 1)))
    fn = int(np.sum((y_true == 1) & (y_pred == 0)))

    accuracy = (tp + tn) / max(1, len(y_true))
    precision_real = tp / max(1, tp + fp)
    recall_real = tp / max(1, tp + fn)
    f1_real = 2 * precision_real * recall_real / max(1e-9, precision_real + recall_real)

```

```

precision_fake = tn / max(1, tn + fn)
recall_fake = tn / max(1, tn + fp)
f1_fake = 2 * precision_fake * recall_fake / max(1e-9, precision_fake + recall_fake)

precision_macro = (precision_real + precision_fake) / 2
recall_macro = (recall_real + recall_fake) / 2
f1_macro = (f1_real + f1_fake) / 2

order = np.argsort(y_score)
s = y_score[order]
y = y_true[order]
pos = int(np.sum(y == 1))
neg = len(y) - pos
auc = float("nan")
if pos > 0 and neg > 0:
    ranks = np.empty(len(y), dtype=np.float64)
    i = 0
    while i < len(y):
        j = i
        while j + 1 < len(y) and s[j + 1] == s[i]:
            j += 1
        ranks[i : j + 1] = (i + j) / 2.0 + 1.0
        i = j + 1
    auc = float((ranks[y == 1].sum() - pos * (pos + 1) / 2.0) / (pos * neg))

return {
    "accuracy": accuracy,
    "confusion_matrix": np.array([[tn, fp], [fn, tp]]),
    "real": {"precision": precision_real, "recall": recall_real, "f1": f1_real},
    "fake": {"precision": precision_fake, "recall": recall_fake, "f1": f1_fake},
    "macro": {"precision": precision_macro, "recall": recall_macro, "f1": f1_macro},
    "roc_auc": auc,
}

def main():
    print("Loading model...")
    model = load_model(MODEL_PATH)
    print("Loading CIFAKE test split...")
    ds = load_dataset(DATASET_ID, split="test")

    y_true = np.asarray(ds["label"], dtype=int)
    n = len(y_true)
    n_real = int(np.sum(y_true == 1))
    n_fake = n - n_real
    print(f"Test samples: {n} ({n_real} Real, {n_fake} Fake)")

    probs = np.zeros(n, dtype=np.float32)
    for i in range(0, n, BATCH_SIZE):
        batch = ds[i : i + BATCH_SIZE]["image"]
        arrs = np.stack([np.asarray(img.convert("RGB"), dtype=np.float32) for img in batch])
        if RESCALE:
            arrs /= 255.0
        preds = model.predict(arrs, verbose=0, batch_size=BATCH_SIZE)
        probs[i : i + len(batch)] = preds[:, 0]
        print(f"\r Predicting {min(i + len(batch), n):,}/{n:,}", end="", flush=True)
    print()

    metrics = compute_metrics(y_true, probs)
    cm = metrics["confusion_matrix"]

    print("\n" + "=" * 50)
    print("CIFAKE EVALUATION - my_model21.h5")
    print("=" * 50)
    print(f"Test samples : {n} (Real={n_real}, Fake={n_fake})")
    print(f"Accuracy      : {metrics['accuracy']:.4f} ({metrics['accuracy']*100:.2f}%)")
    print(f"ROC-AUC       : {metrics['roc_auc']:.4f}")
    print()

```

```

print("Confusion matrix (rows=actual, cols=predicted):")
print("                Pred Fake   Pred Real")
print(f"  Actual Fake      {cm[0,0]:6d}      {cm[0,1]:6d}")
print(f"  Actual Real      {cm[1,0]:6d}      {cm[1,1]:6d}")
print()
print(f"{'Class':<8}{'Precision':>12}{'Recall':>10}{'F1':>10}")
for name in ("real", "fake", "macro"):
    m = metrics[name]
    print(f"{name.capitalize():<8}{m['precision']:>12.4f}{m['recall']:>10.4f}{m['f1']:>10.4f}")

if __name__ == "__main__":
    """Smoke tests for the Flask app (stdlib unittest).

    Run with: .\\.venv\\Scripts\\python.exe smoke_test.py
    Note: importing `app` loads TensorFlow, so the suite takes a few seconds.
    The predict test exercises lazy model loading.
    """

    import io
    import unittest

    import numpy as np
    from PIL import Image

    import app

    def make_png(size=32):
        buf = io.BytesIO()
        Image.fromarray(np.zeros((size, size, 3), dtype=np.uint8)).save(buf, "PNG")
        buf.seek(0)
        return buf

    class AppTestCase(unittest.TestCase):
        def setUp(self):
            self.client = app.app.test_client()

        def test_index_ok(self):
            r = self.client.get("/")
            self.assertEqual(r.status_code, 200)

        def test_reject_missing_file(self):
            r = self.client.post("/predict", data={})
            self.assertEqual(r.status_code, 400)

        def test_reject_bad_type(self):
            r = self.client.post("/predict", data={"file": (io.BytesIO(b"x"), "notes.txt")})
            self.assertEqual(r.status_code, 400)

        def test_reject_too_large(self):
            big = io.BytesIO(b"\0" * (6 * 1024 * 1024))
            r = self.client.post("/predict", data={"file": (big, "big.png")})
            self.assertEqual(r.status_code, 413)

        def test_predict_ok(self):
            r = self.client.post("/predict", data={"file": (make_png(), "img.png")})
            self.assertEqual(r.status_code, 200)
            data = r.get_json()
            self.assertIn(data["label"], ("Real", "Fake"))
            self.assertIn("confidence", data)
            self.assertIn("probability", data)

    if __name__ == "__main__":

```

```
unittest.main(verbosity=2)

tensorflow==2.21.0
flask==3.1.3
pillow==12.3.0
numpy==2.5.1
datasets==5.0.1
```

LAMPIRAN A

A Struktur & Menjalankan

```
# Masuk direktori proyek
# Python 3.12 diperlukan (TensorFlow belum dukung 3.14)
uv python install 3.12
uv venv --python 3.12 .venv
uv pip install -r requirements.txt --python .venv/bin/python
.venv/bin/python app.py # http://127.0.0.1:5000
.venv/bin/python smoke_test.py # 5 pengujian OK
.venv/bin/python eval_cifake.py # accuracy 93,95%, ROC-AUC 0,9861
```

Catatan: versi Python sistem adalah 3.14; gunakan .venv/bin/python (3.12). TensorFlow berjalan pada CPU; peringatan CUDA/cuInit dapat diabaikan.

Lampiran B API Reference

Endpoint	Metode	Respons	Keterangan
/	GET	text/html	Antarmuka index.html
/predict	POST multipart	{label, probability, confidence}	200 OK; 400; 413 >5MB
(error)	—	{error: ...}	Tidak ada berkas / jenis / gagal / ukuran

— Akhir Buku Panduan —

CIFAKE Detector • Hamid Albar Nurrasyid 24060120130069 • Departemen Informatika, FSM, Universitas Diponegoro 2026